Git Workflow

Version: 1.0

Purpose

This document defines the Git workflow for Trading Platform Pro and its primary application, the Trading Cockpit.
The goal is to keep the repository:
• clean
• reviewable
• traceable
• reversible
• synchronized with documentation
• protected by automated quality gates
Git history shall represent coherent product and technical changes.

Workflow Principles

Every commit should be:
• atomic
• reviewable
• traceable
• reversible
• coherent
Never mix unrelated changes.
A commit may contain code, tests and documentation when they belong to the same logical change.
Prefer one complete coherent change over artificially separating required implementation artifacts.

Standard Workflow

```text
Analyze
↓
Implement
↓
Add or Update Tests
↓
Update Documentation
↓
Run Focused Tests
↓
Run Required Regression Tests
↓
Run Ruff
↓
Validate Documentation Generation
↓
Review Complete Diff
↓
git add
↓
git commit
↓
```

```
git push
```

Documentation generation is required when Markdown documentation changes.
Steps that do not apply to a change may be omitted only when their scope is genuinely unaffected.

Daily Development Workflow

For every task:
1. Understand the requirement.
2. Analyze the affected implementation.
3. Review affected documentation.
4. Identify architecture and capability ownership.
5. Implement the smallest meaningful change.
6. Add or update automated tests.
7. Update affected documentation.
8. Execute focused tests.
9. Execute required regression tests.
10. Verify formatting and linting.
11. Validate documentation generation where required.
12. Review the complete diff.
13. Stage the coherent change.
14. Commit.
15. Push.

Do not commit before the implementation and its directly affected documentation are synchronized.

Current Repository State

Before starting a change inspect the current repository state.
Use:
```bash
git status
```

Review:
• modified files
• staged files
• untracked files
• current branch

Do not assume the working tree is clean.
Existing unrelated local changes shall be identified before implementation.
Do not overwrite or discard unrelated user changes.

Diff Review

Review the complete change before staging or committing.
Use where appropriate:
```bash
git diff
```

For staged changes:
```bash
git diff --staged
```

The review shall verify:
• intended files changed
• no unrelated changes included
• no debug code remains

• no temporary code remains
• no secrets were added
• tests match the implementation
• documentation matches the implementation
For trading-critical changes additionally review:
• external side effects
• order lifecycle changes
• duplicate submission risk
• retry behaviour
• PAPER and LIVE impact
• reconciliation impact

Branch Strategy

Default branch:
```text
main
```
Development work shall use dedicated branches.
Direct commits to `main` are not part of the normal development workflow.
Branch categories may include:
```text
feature/
fix/
refactor/
docs/
test/
release/
hotfix/
```
Use only the branch type that matches the change.

Branch Naming Convention

Branch names shall use lowercase descriptive identifiers.
Examples:
```text
feature/order-workflow
feature/trading-candidate-review
fix/broker-acknowledgement-state
fix/duplicate-submission-guard
refactor/market-data-adapter
docs/runtime-lifecycle
test/order-reconciliation
release/1.0.0
hotfix/live-order-validation
```
Avoid vague branch names such as:
```text
feature/update
fix/stuff
test/test
```
Branch names should communicate the primary change.

Branch Scope

One branch should focus on one coherent objective.
A branch may contain multiple commits when they contribute to the same objective.
Avoid combining unrelated:
• features
• bug fixes
• refactorings
• documentation topics
in one branch.
Large objectives should be divided into reviewable stages where practical.

Creating a Branch

Before creating a branch:
1. Verify the current branch.
2. Review `git status`.
3. Ensure unrelated local changes are understood.
4. Update the local base branch according to the team workflow.
5. Create the dedicated branch.
Example:
```bash
git switch main
git pull
git switch -c feature/order-workflow
```
Do not blindly switch branches when uncommitted changes may be affected.

Commit Messages

Commit messages shall be concise and descriptive.
Prefer imperative or clear change-oriented wording.
Examples:
```text
Add duplicate submission guard
Preserve broker acknowledgement state
Improve market data freshness handling
Add reconciliation regression tests
Update runtime lifecycle documentation
Validate generated documentation in CI
```
Avoid:
```text
fix
update
changes
misc
stuff
wip
```
The commit message should explain the primary change without requiring inspection of every file.

Commit Message Scope

A project-wide mandatory conventional commit format is not required unless explicitly adopted.

Documentation commits may use:
```text
docs: update runtime lifecycle
```
Tests may use:
```text
test: add order rejection regression
```
The selected wording shall remain clear and consistent.
Do not introduce a complex commit-message convention without a concrete workflow requirement.

Recommended Commit Size

One commit should represent one logical change.
Good examples:
• implement one workflow capability
• fix one defect
• refactor one component
• add one regression scenario
• update one coherent documentation area
A logical implementation commit may include:
• source code
• affected tests
• affected Markdown documentation
when all files are required for the same change.

Code and Documentation in One Commit

Do not separate required documentation from the implementation solely because one file is code and another is Markdown.
Example:
```text
Order acknowledgement behaviour changed
↓
Order workflow code updated
↓
Regression tests updated
↓
API_Guidelines.md updated
```
These changes may form one coherent commit.
Separate documentation commits are appropriate when the documentation change is independent from code behaviour.

Generated Documentation

Markdown under:
```text
docs/
```
is the documentation source of truth.
Generated files under:
```text
docs/generated/docx/
docs/generated/pdf/
```

are derived artifacts.
Generated DOCX and PDF files shall not be edited manually.
Repository handling of generated artifacts shall follow the documented project policy.
Do not treat generated files as independent source changes.

Documentation Generation

When Markdown documentation changes, run:
```bash
python scripts/generate_docs.py
```

The command shall complete successfully before commit.
Documentation validation uses Markdown source.
Generated DOCX and PDF files are not re-read for content validation.

Before Commit

Before committing verify:
• requirement implemented
• focused tests passed
• required regression tests passed
• `ruff check .` passed
• `ruff format --check .` passed
• documentation updated
• documentation generation passed where required
• complete diff reviewed
• no debug code remains
• no commented-out temporary code remains
• no temporary files are staged
• no secrets are staged
• no credentials are staged
For trading-critical changes additionally verify:
• order side effects reviewed
• duplicate submission risk reviewed
• retry behaviour reviewed
• PAPER and LIVE impact reviewed
• reconciliation impact reviewed

Staging

Stage only the files belonging to the coherent change.
Prefer explicit staging where the working tree contains unrelated changes.
Example:
```bash
git add app/application/orders/
git add tests/
git add docs/
```

Review staged changes:
```bash
git diff --staged
```

Do not use broad staging blindly when unrelated changes exist.

Partial Staging

Partial staging may be used when one file contains unrelated changes.
Example:
```bash
git add -p
```
Use partial staging carefully.
A partially staged implementation shall still represent a complete coherent commit.
Do not create commits that knowingly leave the repository in an invalid intermediate state unless an explicitly documented migration workflow requires it.

Commit

Create the commit only after staged changes have been reviewed.
Example:
```bash
git commit -m "Add duplicate submission guard"
```
A successful commit does not replace test or quality verification.
Do not commit known unresolved trading-critical behaviour as completed work.

Push Policy

Push only after:
• local verification completed
• staged diff reviewed
• commit created successfully
Example:
```bash
git push
```
For a new branch:
```bash
git push -u origin feature/order-workflow
```
A successful push does not replace CI validation.

Pull Requests

Every Pull Request should:
• solve one coherent objective
• have a clear title
• explain the change
• explain relevant behaviour impact
• remain reasonably small
• pass required quality gates
For trading-critical changes the Pull Request should identify where relevant:
• order workflow impact
• duplicate submission risk
• retry behaviour
• broker acknowledgement semantics
• PAPER and LIVE impact
• reconciliation impact
Do not hide business-critical behaviour changes inside generic refactoring descriptions.

Pull Request Description

A Pull Request description should communicate:
1. What changed?
2. Why was the change required?
3. Which capability owns the change?
4. How was the change tested?
5. Which documentation changed?
6. Are there operational impacts?
For trading-critical changes additionally document relevant external side effects and safety considerations.

Pull Request Quality Gates

Before merge, required CI quality gates shall pass.
Potential required gates include:
• Ruff check
• Ruff format verification
• Unit tests
• Integration tests
• Trading workflow regression tests
• Architecture boundary checks
• Documentation source validation
• Documentation generation validation
Only implemented and configured gates are enforced by CI.
Do not describe a planned quality gate as active until it exists.

Review Policy

Merge only after the required review process is complete.
Review shall evaluate:
• requirement correctness
• architecture boundaries
• capability ownership
• code quality
• tests
• documentation
• external side effects
• security impact
Trading-critical changes require explicit review of:
• state transitions
• duplicate submission risk
• retry behaviour
• PAPER and LIVE impact
• reconciliation impact

Merge Policy

• Do not commit directly to `main` during normal development.
• Work in dedicated branches.
• Merge only after required review.
• Merge only after required CI gates pass.
• Keep commits coherent and reviewable.
Emergency hotfix handling may use a dedicated `hotfix/` branch and an expedited review process.
An expedited process shall not bypass trading-critical safety validation.

Merge Strategy

The project may use:
• merge commits
• squash merge
• rebase merge
The selected repository strategy shall be applied consistently.
The strategy should preserve useful history and reviewability.
Do not rewrite shared branch history without a concrete reason.

Repository Hygiene

Do not commit:
• developer-local IDE settings unless project-wide
• temporary files
• log files
• cache files
• local virtual environments
• local databases unless explicitly required as fixtures
• secrets
• credentials
• private keys
Generated artifacts shall follow the explicit repository policy.
Do not assume every generated file is automatically ignored.

Secrets

Before commit inspect staged changes for:
• API keys
• passwords
• access tokens
• broker credentials
• private keys
• secret environment variables
Secrets shall never be committed.
If a secret is committed:
1. Treat the secret as exposed.
2. Revoke or rotate the secret.
3. Remove the secret from the repository.
4. Evaluate history cleanup.
5. Document the incident where required.
Deleting the secret in a later commit does not make the original secret safe.

Temporary and Debug Code

Do not commit:
• temporary `print(...)` diagnostics
• disabled production logic
• commented-out experimental blocks
• temporary file paths
• local machine assumptions
Intentional diagnostic capabilities shall use the project logging infrastructure and explicit configuration.

Recovery

If a mistake is committed:

1. Identify the problem.
2. Determine whether the commit was pushed.
3. Determine whether the branch is shared.
4. Create a dedicated corrective commit when history is already shared.
5. Avoid rewriting shared history unless explicitly required.
For local unshared commits, history cleanup may be used carefully.
Do not use destructive Git commands without understanding the affected changes.

Revert

Use a revert when a shared commit must be undone while preserving repository history.
Example:
```bash
git revert
```
Review the revert diff before pushing.
A revert of a trading-critical change shall include the same safety and regression review as the original capability.

Force Push

Avoid force push on shared branches.
If history rewriting is explicitly required, prefer:
```bash
git push --force-with-lease
```
over unrestricted force push.
Never rewrite `main` history as part of normal development.

Conflict Resolution

Merge conflicts shall be resolved by understanding both changes.
Do not automatically choose:
```text
ours
```
or:
```text
theirs
```
for business-critical files without review.
After resolving conflicts:
1. Review the resulting file.
2. Run affected tests.
3. Run required regression tests.
4. Regenerate documentation where Markdown changed.
5. Review the final diff.

Documentation Commits

Documentation changes should be grouped by coherent topic.
Examples:
```text
docs: align runtime and monitoring lifecycle
docs: update trading cockpit architecture
```

docs: document documentation generation workflow
```

Independent documentation cleanup should use a documentation-focused commit.
Documentation required by a code contract change may remain in the same implementation commit.
---
# Test Commits

Tests may be committed with the implementation they verify.
A dedicated test commit is appropriate when:
• adding independent regression coverage
• improving test infrastructure
• reorganizing test structure
Do not separate a required bug regression test from the bug fix solely to create smaller file groups.
---
# Refactoring Commits

Refactoring commits should preserve intended behaviour.
Avoid combining large refactoring with unrelated feature behaviour.
When a refactoring changes behaviour intentionally:
• identify the behaviour change
• update tests
• update documentation where affected
• describe the change clearly in the commit or Pull Request
---
# Release Branches

Release branches may use:
```text
release/
```

Example:
```text
release/1.0.0
```

Release branches shall only be introduced when the release workflow requires them.
Do not maintain permanent release branches without a concrete delivery need.

Versioning

Use the project versioning strategy documented for releases.
Semantic Versioning may be used:
```text
MAJOR.MINOR.PATCH
```

Examples:
```text
1.0.0
1.1.0
1.1.1
```

Versioning policy shall remain synchronized with the actual release workflow.
Do not increment versions for ordinary development commits unless the release process requires it.

```

Changelog

Update `CHANGELOG.md` according to the release workflow.
The changelog should describe meaningful user-facing or operational changes.
Avoid filling the changelog with every internal formatting or refactoring commit.
Trading-critical behaviour changes should be represented clearly.

CI Failure After Push

If CI fails after push:
1. Identify the failed quality gate.
2. Review the CI failure output.
3. Reproduce the failure locally where practical.
4. Correct the root cause.
5. Run relevant local verification.
6. Review the correction diff.
7. Commit the correction.
8. Push again.

Do not bypass required CI gates solely to merge faster.

Trading-Critical Git Review

Before committing or merging trading-critical changes verify:
• affected workflow identified
• external side effects understood
• order state transitions reviewed
• broker acknowledgement semantics preserved
• duplicate submission risk reviewed
• retry behaviour reviewed
• timeout behaviour reviewed
• disconnect and reconnect impact reviewed
• PAPER and LIVE impact reviewed
• persistence impact reviewed
• reconciliation impact reviewed
• regression tests included
• documentation synchronized

Git workflow is part of trading workflow risk control.

Git Workflow Review Checklist

Before completing a change verify:
• dedicated branch used
• repository state reviewed
• requirement understood
• current implementation inspected
• coherent change implemented
• tests added or updated
• focused tests passed
• required regression tests passed
• Ruff passed
• formatting check passed
• documentation synchronized
• documentation generation passed where required
• complete diff reviewed
• staged diff reviewed

• no unrelated files staged
• no debug code staged
• no temporary files staged
• no secrets staged
• commit is atomic
• commit message is descriptive
• push completed
• CI result reviewed

For trading-critical changes additionally verify:

• external side effects reviewed
• duplicate submission risk reviewed
• retry behaviour reviewed
• PAPER and LIVE impact reviewed
• reconciliation impact reviewed

Related Documents

• Product_Vision.md
• Product_Roadmap.md
• Project_Overview.md
• Architecture.md
• Coding_Standards.md
• Development_Guidelines.md
• Testing_Strategy.md
• CI_CD.md
• Configuration.md
• Runtime.md
• AGENTS.md